



Universidad Veracruzana

Servicios REST, token y CSRF

Conceptos de seguridad en REST

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito

Comprender los conceptos básicos que permiten implementar el endpoint de rastreo de camiones en Django.

Esta presentación introduce:

- qué es un servicio REST;
- qué es un endpoint;
- qué métodos HTTP se usan;
- qué papel tiene el token;
- por qué usamos `@csrf_exempt` en el endpoint de rastreo.

El objetivo no es profundizar en APIs REST avanzadas, sino entender lo necesario para implementar el servicio de rastreo.

Contexto del proyecto

En el proyecto de logística, un camión o un script externo envía su ubicación a la aplicación Django.

```
Camión o script externo
    |
    | HTTP POST + JSON + Token
    v
Endpoint REST en Django
    |
    v
Historial de ubicaciones
```

La aplicación recibe la ubicación, valida los datos y guarda el registro.

¿Qué es un servicio REST?

Un servicio REST permite que una aplicación exponga recursos para que otros sistemas puedan consultarlos o modificarlos mediante HTTP.

Ejemplos de recursos en el proyecto:

Recurso	Posible uso
bienes	Consultar bienes registrados
clientes	Consultar clientes
camiones	Identificar unidades
ubicaciones	Guardar posiciones enviadas por camiones

¿Qué es un endpoint?

Un endpoint es una URL específica donde un sistema puede enviar o solicitar información.

En la práctica de rastreo, el endpoint es:

```
/api/camiones/ubicacion/
```

Ese endpoint recibe solicitudes del camión o script externo.

El endpoint es la puerta de entrada técnica al servicio.

Endpoint de rastreo

El endpoint de rastreo espera una solicitud como esta:

```
POST /api/camiones/ubicacion/
```

Esto significa:

- `POST` : se enviarán datos nuevos;
- `/api/camiones/ubicacion/` : recurso relacionado con la ubicación del camión;
- el cuerpo de la solicitud contendrá datos en JSON;
- la respuesta también será JSON.

Métodos HTTP principales

Método	Uso común	Ejemplo
GET	Consultar información	Obtener historial de ubicaciones
POST	Crear un nuevo registro	Registrar ubicación del camión
PUT	Reemplazar un recurso completo	Actualizar todos los datos de una unidad
PATCH	Modificar parcialmente	Cambiar solo el estatus de un bien
DELETE	Eliminar un recurso	Borrar un registro

Para la práctica de rastreo usamos POST .

¿Por qué usamos POST?

Usamos `POST` porque el camión está enviando datos para crear un nuevo registro en el historial.

```
{  
  "camion_id": "CAM-0001-2026",  
  "latitud": 19.5438,  
  "longitud": -96.9102,  
  "velocidad": 42.5,  
  "registrado_en": "2026-05-24T10:30:00"  
}
```

Cada solicitud válida genera una nueva ubicación guardada.

Partes de una solicitud HTTP

```
POST /api/camiones/ubicacion/ HTTP/1.1  
Host: 127.0.0.1:8000  
Content-Type: application/json  
Authorization: Bearer TOKEN_DE_PRACTICA_CAMIONES
```

```
{  
  "camion_id": "CAM-0001-2026",  
  "latitud": 19.5438,  
  "longitud": -96.9102,  
  "velocidad": 42.5,  
  "registrado_en": "2026-05-24T10:30:00"  
}
```

Elementos importantes

Elemento	Función
Método	Indica la acción solicitada
Ruta	Identifica el endpoint
Content-Type	Indica que el cuerpo viene en JSON
Authorization	Envía el token
Cuerpo JSON	Contiene la ubicación del camión

¿Qué es un token?

Un token es una credencial técnica que permite identificar o autorizar una solicitud.

En la práctica usamos:

```
Authorization: Bearer TOKEN_DE_PRACTICA_CAMIONES
```

El servidor revisa si el token recibido coincide con el token esperado.

El token permite que el endpoint no acepte solicitudes anónimas o no autorizadas.

Token en la práctica de rastreo

En esta práctica el token será estático.

Esto significa que:

- se define en `settings.py` ;
- el script del camión lo envía en cada solicitud;
- Django lo compara antes de guardar la ubicación.

```
TOKEN_RASTREO_CAMIONES = "TOKEN_DE_PRACTICA_CAMIONES"
```

Alcance del token estático

El token estático se usa para simplificar la práctica.

Permite concentrarse en:

- recibir solicitudes externas;
- leer encabezados HTTP;
- validar una credencial básica;
- validar datos de entrada;
- guardar información en SQLite.

El token estático es útil para aprender el flujo, pero no es suficiente como solución avanzada de producción.

Riesgo de un token estático

Si alguien obtiene el token, podría intentar enviar ubicaciones falsas.

Por ejemplo:

- simular que un camión está en otro lugar;
- generar registros no autorizados;
- alterar la trazabilidad;
- saturar el endpoint con solicitudes.

Esta limitación se debe reconocer como parte del análisis de seguridad.

¿Qué es CSRF?

CSRF significa **Cross-Site Request Forgery**.

Es un ataque donde un sitio malicioso intenta provocar que el navegador de un usuario autenticado envíe una solicitud no deseada a otra aplicación.

Ejemplo conceptual:

```
Usuario con sesión activa
|
| Visita sitio malicioso
v
Sitio malicioso intenta enviar solicitud
|
v
Aplicación donde el usuario tenía sesión
```

CSRF en formularios web

Django protege formularios web usando un token CSRF.

En templates, normalmente se usa:

```
<form method="post">
  {% csrf_token %}
  <!-- campos del formulario -->
  <button type="submit">Guardar</button>
</form>
```

Ese token ayuda a comprobar que la solicitud viene desde un formulario legítimo de la aplicación.

Protección CSRF en Django

Cuando una vista recibe datos desde un formulario web, Django espera que la solicitud incluya el token CSRF.

Esto es común en vistas como:

- alta de bienes;
- edición de bienes;
- formularios de clientes;
- cambios realizados por usuarios con sesión.

CSRF protege solicitudes realizadas desde navegador usando sesión de usuario.

¿Qué hace `@csrf_exempt`?

`@csrf_exempt` indica que una vista no exigirá token CSRF.

Ejemplo:

```
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def registrar_ubicacion(request):
    ...
```

Esto debe usarse con cuidado, porque desactiva una protección importante de Django.

¿Por qué usamos `@csrf_exempt` en rastreo?

El endpoint de rastreo no será usado por un formulario web con sesión de usuario.

Será usado por:

- un camión;
- un dispositivo;
- un script externo;
- otro sistema.

Ese cliente externo no tiene un formulario Django ni un `{% csrf_token %}` .

Por eso usamos `@csrf_exempt` .

Contraste: formulario web vs endpoint API

FORMULARIO WEB

- Lo usa una persona desde navegador.
- Trabaja con sesión de usuario.
- Debe usar `{% csrf\_token %}`.
- No debe usar `@csrf\_exempt` salvo excepción justificada.

ENDPOINT DE RASTREO

- Lo usa un cliente externo.
- No depende de sesión de navegador.
- Usa token en encabezado HTTP.
- Puede usar `@csrf\_exempt` con controles adicionales.

@csrf_exempt no significa sin seguridad

Al usar `@csrf_exempt`, la vista debe compensar con otros controles.

En el endpoint de rastreo se usan:

- token obligatorio;
- solo método `POST` ;
- validación de JSON;
- validación de latitud;
- validación de longitud;
- validación de velocidad;
- manejo controlado de errores.

`@csrf\_exempt` elimina la validación CSRF, no elimina la responsabilidad de proteger el endpoint.

Relación entre @csrf_exempt y token

En el endpoint de rastreo:

```
@csrf_exempt
@require_POST
def registrar_ubicacion(request):
    ...
```

La protección se basa en:

```
Authorization: Bearer TOKEN_DE_PRACTICA_CAMIONES
```

El token sustituye la necesidad de una sesión de navegador para este caso específico.

Flujo completo del endpoint

1. Recibir

El camión envía un POST con JSON.

2. Token

Django revisa el encabezado Authorization.

3. Validar

Se revisan coordenadas, velocidad y fecha.

4. Guardar

Se crea un registro en UbicacionCamion.

Códigos de respuesta esperados

Código	Cuándo ocurre
201	La ubicación se guardó correctamente
400	El JSON o algún dato es inválido
401	El token no existe o es incorrecto
405	Se usó un método distinto a POST
500	Error interno no controlado

El objetivo es devolver respuestas claras y controladas.

Ejemplo con curl

```
curl -X POST http://127.0.0.1:8000/api/camiones/ubicacion/ \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN_DE_PRACTICA_CAMIONES" \
-d '{
  "camion_id": "CAM-0001-2026",
  "latitud": 19.5438,
  "longitud": -96.9102,
  "velocidad": 42.5,
  "registrado_en": "2026-05-24T10:30:00"
}'
```

Respuesta esperada

```
{  
  "mensaje": "Ubicación registrada correctamente.",  
  "ubicacion_id": 1  
}
```

Esta respuesta indica que:

- el token fue aceptado;
- el JSON pudo leerse;
- los datos pasaron las validaciones;
- el registro fue guardado en la base de datos.

Lo que debe quedar claro

- REST organiza servicios mediante recursos y métodos HTTP.
- Un endpoint es una URL que recibe o entrega información.
- `POST` se usa para crear nuevos registros.
- El token autentica técnicamente la solicitud.
- CSRF protege formularios web con sesión.
- `@csrf_exempt` puede usarse en endpoints externos, pero exige otros controles.

Alcance de esta introducción

Esta sesión permite entender la práctica de rastreo.

No profundiza en:

- Django REST Framework;
- JWT;
- OAuth2;
- HMAC;
- tokens temporales;
- protección completa contra repetición de solicitudes;
- diseño avanzado de APIs REST.

Estos temas corresponden a un curso más específico de servicios REST seguros.

Gracias
¿Preguntas?

